

Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп'ютерних систем
Кафедра інформаційних систем

КУРСОВА РОБОТА

з дисципліни «ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ»

Backend-сервіс управління системою музичних фестивалів

Студента 2 курсу AI-244 групи
Спеціальності F3 – «Комп'ютерні
науки» Ткаченка В. А.

(прізвище та ініціали)

М.А. Годовиченко

(посада, вчене звання, науковий ступінь, прізвище та
ініціали)

Національна _____ шкала

Кількість балів: _____

Оцінка: ECTS _____

Члени комісії _____

(підпис)

(прізвище та ініціали)

(підпис)

(прізвище та ініціали)

(підпис)

(прізвище та ініціали)

Одеса – 2026

ЗМІСТ

Вступ	3
Аналіз предметної області	4
1.1 Огляд систем оренди електросамокатів	5
1.2 Аналіз існуючих програмних рішень	6
1.3 Ролі користувачів та сценарії використання системи	7
Проектування програмного забезпечення	8
2.1 Проектування структури даних	8
2.2 Опис архітектури застосунку	9
2.3 Опис REST API	10
2.4 Нормалізація бази даних	11
2.5 Фізична модель та словник даних	12
Реалізація програмного продукту	13
3.1 Моделі даних	13
3.2 Репозиторії	14
3.3 Сервісний шар	14
3.4 Контролери	15
3.5 Конфігурація застосунку	16

3.6 Архітектура та структура пакетів проєкту	17
3.7 Обробка винятків та валідація вхідних даних	18
Тестування та налагодження	19
Висновки	20
Перелік використаних джерел	21
Додаток А	22

Розвиток сервісів мікромобільності та популяризація екологічного транспорту сприяють активному впровадженню систем оренди електросамокатів у сучасних містах. Управління такими системами потребує ефективної обробки даних про користувачів, транспортні засоби, оренди, платежі та технічний стан самокатів. Використання ручного обліку або застарілих програмних рішень може призводити до помилок, втрати даних та зниження якості обслуговування клієнтів.

Метою даної курсової роботи є розробка серверної частини (backend-сервісу) для управління системою оренди електросамокатів з використанням об'єктно-орієнтованого підходу.

Для досягнення мети було поставлено такі завдання:

- провести аналіз предметної області та визначити основні сутності системи;
- спроектувати базу даних та зв'язки між таблицями;
- розробити архітектуру RESTful вебсервісу на базі фреймворку Spring Boot;
- реалізувати CRUD-операції для управління користувачами, електросамокатами та орендами;

- реалізувати механізми пошуку, фільтрації та отримання статистичної інформації;
- провести тестування розробленого програмного забезпечення.

Об'єктом дослідження є процес управління системою оренди електросамокатів.

Предметом дослідження є розробка програмного забезпечення для автоматизації процесів оренди та управління електросамокатами засобами мови Java.



Рисунок 1.1 – Діаграма варіантів використання (Use Case Diagram)

1.3 Огляд та аналіз існуючих програмних рішень

На сучасному ринку сервісів мікромобільності існує низка програмних продуктів, призначених для оренди та управління електросамокатами. Для формування вимог до власної системи було проведено аналіз таких популярних платформ, як Lime, Bolt та Bird.

Платформа Lime є одним із найбільших сервісів прокату електросамокатів у світі та пропонує широкий набір функцій для користувачів і операторів. Проте її недоліками є складна внутрішня архітектура, висока вартість впровадження та обмежений доступ до окремих

компонентів для сторонніх розробників. Сервіс Bolt поєднує оренду електросамокатів з іншими транспортними послугами, однак через універсальність платформи деякі функції управління парком самокатів мають обмежену гнучкість. Платформа Bird забезпечує ефективне керування транспортними засобами, але орієнтована переважно на великі міста та великі операторські мережі, що може бути надлишковим для локальних сервісів прокату.

З огляду на виявлені недоліки існуючих рішень, розробка власного RESTful API є обґрунтованою. Створювана система буде спеціалізованою, легкою для інтеграції з вебзастосунками та мобільними клієнтами, а також орієнтованою на ефективне управління користувачами, електросамокатами, орендами та статистикою використання транспорту.

Розроблений backend-сервіс дозволить автоматизувати основні бізнес-процеси системи оренди електросамокатів, забезпечити надійне зберігання даних та спростити взаємодію між клієнтськими застосунками й серверною частиною системи.

1.4 Ролі користувачів та сценарії використання системи

Для забезпечення безпеки та правильного розподілу функцій у системі передбачено декілька рівнів доступу. Інформаційна система повинна підтримувати взаємодію з трьома основними типами користувачів: Адміністратор системи, Оператор сервісу та Користувач.

Адміністратор системи має найвищий рівень привілеїв. До його сценаріїв використання входить: управління обліковими записами користувачів та операторів, моніторинг стану системи за допомогою метрик Actuator, налаштування параметрів сервісу, перегляд статистики використання електросамокатів та вирішення критичних помилок.

Оператор сервісу відповідає за управління парком електросамокатів. Його бізнес-процеси включають: додавання нових електросамокатів до системи, оновлення інформації про транспортні засоби, контроль їх технічного стану, виведення самокатів з експлуатації для ремонту та перегляд інформації про активні оренди.

Користувач є кінцевим користувачем системи, права якого обмежені функціями оренди. Через клієнтський застосунок, що взаємодіє з розробленим API, користувач може переглядати список доступних електросамокатів, виконувати пошук за різними параметрами, розпочинати

та завершувати оренду, переглядати історію власних поїздок і отримувати інформацію про вартість використання сервісу.

ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Проєктування структури даних

На основі аналізу предметної області було спроектовано реляційну модель даних. Зв'язки між сутностями реалізовані наступним чином:

- User та Rental мають відношення «один до багатьох» (1), оскільки один користувач може здійснювати багато оренд, а кожна оренда належить лише одному користувачу.
- Scooter та Rental мають відношення «один до багатьох» (1), оскільки один електросамокат може бути орендований багато разів у різний час, але кожна оренда пов'язана лише з одним електросамокатом.
- User та Scooter мають відношення «багато до багатьох» (M), яке реалізується через проміжну сутність Rental. Користувач може орендувати різні електросамокати, а один електросамокат може використовуватися багатьма користувачами.
- Scooter та Maintenance мають відношення «один до багатьох» (1), оскільки кожен електросамокат може проходити декілька технічних обслуговувань або ремонтів протягом свого життєвого циклу.
- Rental та Payment мають відношення «один до одного» (1:1), оскільки кожна завершена оренда супроводжується відповідним платежем за використання електросамоката.

Така структура бази даних забезпечує цілісність інформації, підтримує ефективне виконання CRUD-операцій та дозволяє реалізувати аналітичні запити щодо використання електросамокатів, активності користувачів і статистики оренд.

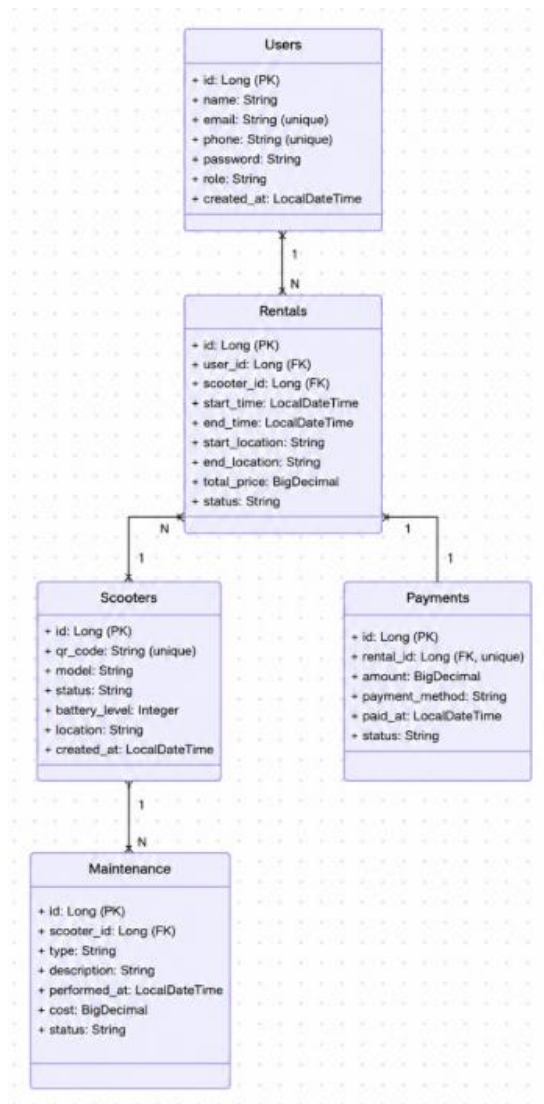


Рисунок 2.1 – UML-діаграма класів сутностей

2.2 Опис архітектури застосунку

Застосунок побудований за класичною трирівневою архітектурою з використанням патерну MVC (Model-View-Controller) для створення REST API системи оренди електросамокатів.

– **Рівень доступу до даних (Repository)** – використовує Spring Data JPA для взаємодії з базою даних PostgreSQL. Репозиторії забезпечують виконання CRUD-операцій для сутностей користувачів, електросамокатів, оренд, платежів та технічного обслуговування.

– **Рівень бізнес-логіки (Service)** – містить сервісні класи, які реалізують основні процеси системи: створення та завершення оренди, перевірку доступності електросамокатів, розрахунок вартості поїздки, обробку платежів і формування статистичних даних. Також на цьому рівні виконується перевірка бізнес-правил та валідація даних.

– **Рівень представлення (Controller)** – обробляє HTTP-запити від клієнтських застосунків, передає їх сервісному рівню та повертає результати у форматі JSON. Контролери надають REST API для управління користувачами, електросамокатами, орендами, платежами та отримання аналітичної інформації.

Така архітектура забезпечує модульність, масштабованість та простоту підтримки програмного забезпечення, а також дозволяє легко інтегрувати систему з веб- та мобільними застосунками.

2.3 Опис REST API

Архітектура API побудована відповідно до принципів REST та призначена для забезпечення взаємодії між клієнтськими застосунками й серверною частиною системи оренди електросамокатів. Усі ресурси доступні за стандартизованими URL-адресами з використанням відповідних HTTP-методів.

API забезпечує управління користувачами, електросамокатами, орендами, платежами та технічним обслуговуванням транспорту. Для виконання операцій використовуються методи GET, POST, PUT та DELETE, що відповідають принципам CRUD.

Детальний перелік реалізованих кінцевих точок (endpoint) наведено у таблиці 2.1.

Таблиця 2.1 – Перелік кінцевих точок REST API

Метод	Endpoint	Опис
GET	/scooters	Отримати список електросамокатів
POST	/scooters	Додати новий електросамокат
GET	/scooters/{id}	Отримати інформацію про електросамокат
PUT	/scooters/{id}	Оновити дані електросамоката
DELETE	/scooters/{id}	Видалити електросамокат
GET	/scooters/available	Отримати список доступних електросамокатів
GET	/scooters/search?query=	Пошук електросамокатів
POST	/users	Створити нового користувача
GET	/users	Отримати список користувачів
GET	/users/{id}	Отримати інформацію про користувача
PUT	/users/{id}	Оновити дані користувача

Продовження таблиці 2.1

Метод	Endpoint	Опис
DELETE	/users/{id}	Видалити користувача
GET	/users/{id}/rentals	Отримати історію оренд користувача
POST	/rentals	Створити нову оренду
GET	/rentals	Отримати список оренд
GET	/rentals/{id}	Отримати інформацію про оренду
PUT	/rentals/{id}	Оновити оренду
DELETE	/rentals/{id}	Видалити оренду
POST	/rentals/{id}/finish	Завершити оренду
POST	/payments	Створити платіж
GET	/payments	Отримати список платежів
GET	/payments/{id}	Отримати інформацію про платіж
PUT	/payments/{id}	Оновити платіж
DELETE	/payments/{id}	Видалити платіж
POST	/maintenance	Додати запис про технічне обслуговування
GET	/maintenance	Отримати список технічних робіт
GET	/maintenance/{id}	Отримати інформацію про технічне обслуговування

Кінець таблиці 2.1

Метод	Endpoint	Опис
PUT	/maintenance/{id}	Оновити запис технічного обслуговування
DELETE	/maintenance/{id}	Видалити запис технічного обслуговування
GET	/analytics/scooters/count	Загальна кількість електросамокатів
GET	/analytics/users/count	Кількість користувачів
GET	/analytics/rentals/count	Кількість оренд
GET	/analytics/payments/total	Загальна сума платежів
GET	/analytics/popular-scooters	Найпопулярніші електросамокати
GET	/analytics/active-rentals	Кількість активних оренд
GET	/actuator/health	Перевірка стану сервісу
GET	/actuator/metrics	Список доступних метрик
GET	/actuator/prometheus	Метрики для систем моніторингу
GET	/scooters/{id}/maintenance	Історія технічного обслуговування електросамоката
GET	/users/{id}/payments	Історія платежів користувача

2.3 Нормалізація бази даних

Для забезпечення цілісності даних, уникнення надмірності та аномалій під час виконання операцій додавання, оновлення або видалення (CRUD), розроблену реляційну базу даних системи оренди електросамокатів було приведено до третьої нормальної форми (3НФ).

На етапі приведення до першої нормальної форми (1НФ) було забезпечено атомарність усіх атрибутів. Кожне поле таблиць містить лише одне неподільне значення. Наприклад, модель електросамоката, його місцезнаходження та рівень заряду батареї зберігаються в окремих полях, а інформація про оренди, платежі та технічне обслуговування винесена до окремих таблиць.

Для відповідності другій нормальній формі (2НФ) у кожній таблиці було створено первинний сурогатний ключ (id) типу BIGINT з автоінкрементом. Усі неключові атрибути повністю залежать від первинного ключа. Наприклад, у таблиці оренд (Rental) дата початку, дата завершення, статус оренди та вартість поїздки залежать виключно від ідентифікатора конкретної оренди.

Приведення до третьої нормальної форми (3НФ) полягало в усуненні транзитивних залежностей. Неключові атрибути не залежать від інших неключових атрибутів. Інформація про користувачів, електросамокати, оренди, платежі та технічне обслуговування зберігається в окремих таблицях, які пов'язані між собою зовнішніми ключами (Foreign Keys) із налаштуванням правил каскадного оновлення та видалення. Це забезпечує узгодженість даних та виключає їх дублювання. Наприклад, при зміні інформації про електросамокат оновлення виконується лише в таблиці Scooter, а всі пов'язані записи оренд і технічного обслуговування автоматично використовують актуальні дані.

Використання третьої нормальної форми дозволяє підвищити ефективність роботи бази даних, спростити супровід системи та забезпечити коректне виконання бізнес-процесів оренди електросамокатів.

2.4 Фізична модель та словник даних

Для фізичної реалізації бази даних обрано реляційну СУБД PostgreSQL.

Структура ключових відношень (таблиць) виглядає наступним чином:

Таблиця «users» (Користувачі) зберігає інформацію про зареєстрованих користувачів системи. Вона містить поле id (первинний ключ, BIGINT), name (ім'я користувача, VARCHAR, обов'язкове поле), email (електронна адреса, VARCHAR, унікальне значення), phone (номер телефону, VARCHAR, унікальне значення), password (хеш пароля, VARCHAR) та role (роль користувача в системі, VARCHAR).

Таблиця «scooters» (Електросамокати) призначена для збереження інформації про транспортні засоби. Включає id (первинний ключ), qr_code

(унікальний QR-код самоката, VARCHAR), model (модель електросамоката, VARCHAR), status (статус доступності, VARCHAR), battery_level (рівень заряду батареї, INTEGER) та location (поточне місцезнаходження, VARCHAR).

Таблиця «rentals» (Оренди) використовується для обліку поїздок користувачів. Вона містить id, зовнішні ключі user_id та scooter_id, дату й час початку оренди start_time, дату й час завершення оренди end_time (типу TIMESTAMP), а також поля total_price (загальна вартість поїздки) та status (статус оренди).

Таблиця «payments» (Платежі) зберігає інформацію про оплату послуг. Містить id, зовнішній ключ rental_id, amount (сума платежу, DECIMAL), payment_method (спосіб оплати, VARCHAR), paid_at (дата та час оплати) і status (статус платежу).

Таблиця «maintenance» (Технічне обслуговування) призначена для обліку ремонтів та сервісного обслуговування електросамокатів. Включає id, зовнішній ключ scooter_id, type (тип технічної роботи, VARCHAR), description (опис виконаних робіт, TEXT), performed_at (дата проведення обслуговування), cost (вартість робіт, DECIMAL) та status (стан виконання робіт).

Зв'язки між таблицями реалізовані за допомогою зовнішніх ключів (Foreign Keys), що забезпечують цілісність даних та підтримують коректну роботу системи оренди електросамокатів.

РЕАЛІЗАЦІЯ ПРОГРАМНОГО ПРОДУКТУ

3.1 Моделі даних

Для взаємодії з базою даних використовуються класи-сутності (Entities), анотовані засобами JPA (@Entity, @Table, @Id). Нижче наведено фрагмент коду для сутності Scooter:

```
@Entity
@Table(name = "scooters")
public class Scooter {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

private String qrCode;

private String model;

private String status;

private Integer batteryLevel;

private String location;

@OneToMany(mappedBy = "scooter")
private List<Rental> rentals;

@OneToMany(mappedBy = "scooter")
private List<Maintenance> maintenanceRecords;

// Геттери, сеттери та конструктори
}

```

Сутність `Scooter` є однією з ключових у системі оренди електросамокатів. Вона містить інформацію про транспортний засіб, включаючи унікальний QR-код, модель, поточний статус, рівень заряду батареї та місцезнаходження. Також сутність пов'язана з таблицями оренд (`Rental`) та технічного обслуговування (`Maintenance`) через зв'язок типу «один до багатьох» (`One-to-Many`).

Завдяки використанню механізму ORM (`Object-Relational Mapping`) фреймворку `Spring Data JPA` забезпечується автоматичне перетворення об'єктів `Java` у записи реляційної бази даних `PostgreSQL` та навпаки.

3.2 Репозиторії

Завдяки використанню `Spring Data JPA`, доступ до даних реалізується через інтерфейси, що наслідують `JpaRepository`. Це дозволяє уникнути написання шаблонного SQL-коду та автоматично отримати базові CRUD-операції для роботи з базою даних. Крім стандартних методів, реалізовано додаткові запити для пошуку електросамокатів та отримання статистичної інформації.

```
@Repository
```

```

public interface ScooterRepository extends JpaRepository<Scooter, Long> {

    List<Scooter> findByModelContainingIgnoreCase(String query);

    List<Scooter> findByStatus(String status);

    @Query("SELECT COUNT(s) FROM Scooter s")
    long countTotalScooters();
}

```

Інтерфейс `ScooterRepository` забезпечує роботу з таблицею електросамокатів. Метод `findByModelContainingIgnoreCase()` використовується для пошуку електросамокатів за моделлю без урахування реєстру символів. Метод `findByStatus()` дозволяє отримувати список самокатів за їх поточним статусом (доступний, орендований, на обслуговуванні тощо). Метод `countTotalScooters()` використовується для формування статистики щодо загальної кількості транспортних засобів у системі.

Використання Spring Data JPA значно спрощує реалізацію рівня доступу до даних та забезпечує високу продуктивність роботи з базою даних PostgreSQL.

3.3 Сервіси та бізнес-логіка

Сервісний шар відповідає за обробку даних та реалізацію бізнес-логіки системи оренди електросамокатів. Для забезпечення коректної взаємодії з базою даних сервіси анотовані як `@Service`, а транзакційність операцій забезпечується анотацією `@Transactional`.

```

@Service
@Transactional
public class ScooterService {

    private final ScooterRepository scooterRepository;

    public ScooterService(ScooterRepository scooterRepository) {
        this.scooterRepository = scooterRepository;
    }

    public List<Scooter> getAllScooters() {
        return scooterRepository.findAll();
    }
}

```

```

public Scooter saveScooter(Scooter scooter) {
    return scooterRepository.save(scooter);
}

public Optional<Scooter> getScooterById(Long id) {
    return scooterRepository.findById(id);
}

public void deleteScooter(Long id) {
    scooterRepository.deleteById(id);
}
}

```

Клас `ScooterService` реалізує основні бізнес-процеси, пов'язані з управлінням електросамокатами. Сервіс надає можливість отримувати список усіх транспортних засобів, додавати нові електросамокати до системи, знаходити самокат за ідентифікатором та видаляти записи з бази даних.

У реальному проєкті сервісний шар також може містити додаткову логіку перевірки доступності електросамокатів, контролю рівня заряду батареї, обробки процесу оренди, розрахунку вартості поїздок та формування статистичних звітів.

3.4 Контролери

Обробка HTTP-запитів реалізована за допомогою класів, позначених анотацією `@RestController`. Контролери використовують механізм ін'єкції залежностей для підключення сервісів та забезпечують взаємодію клієнтських застосунків із серверною частиною системи оренди електросамокатів.

```

@RestController
@RequestMapping("/scooters")
public class ScooterController {

    private final ScooterService scooterService;

    public ScooterController(ScooterService scooterService) {
        this.scooterService = scooterService;
    }
}

```

```

@GetMapping
public ResponseEntity<List<Scooter>> getAll() {
    return ResponseEntity.ok(scooterService.getAllScooters());
}

@PostMapping
public ResponseEntity<Scooter> create(@RequestBody Scooter scooter) {
    return ResponseEntity.ok(scooterService.saveScooter(scooter));
}
}

```

Клас `ScooterController` відповідає за обробку HTTP-запитів, пов'язаних з управлінням електросамокатами. Контролер надає REST API для отримання списку всіх електросамокатів та додавання нових записів до бази даних.

У межах проекту також реалізовані окремі контролери для роботи з користувачами (`UserController`), орендами (`RentalController`), платежами (`PaymentController`) та технічним обслуговуванням (`MaintenanceController`). Вони забезпечують виконання CRUD-операцій та взаємодію між клієнтськими застосунками й сервером через HTTP-запити та відповіді у форматі JSON.

3.5 Конфігурація застосунку

Налаштування підключення до бази даних PostgreSQL та активація метрик Actuator здійснюється у конфігураційному файлі `application.properties`. Для системи оренди електросамокатів використовується окрема база даних `scooter_rental_db`.

```

spring.datasource.url=jdbc:postgresql://localhost:5432/scooter_rental_db
spring.datasource.username=postgres
spring.datasource.password=root

```

```

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

```

```

management.endpoints.web.exposure.include=health,metrics,prometheus

```

Параметр `spring.datasource.url` визначає адресу підключення до бази даних PostgreSQL, у якій зберігаються дані про користувачів, електросамокати, оренди, платежі та технічне обслуговування.

Параметри `spring.datasource.username` та `spring.datasource.password` задають облікові дані для підключення до сервера баз даних.

Властивість `spring.jpa.hibernate.ddl-auto=update` забезпечує автоматичне оновлення структури таблиць відповідно до змін у JPA-сутностях без втрати наявних даних.

Параметр `spring.jpa.show-sql=true` дозволяє відображати SQL-запити у консолі під час виконання застосунку, що спрощує процес налагодження та тестування.

Налаштування `management.endpoints.web.exposure.include` активує точки доступу Spring Boot Actuator для моніторингу працездатності сервісу, збору метрик та інтеграції із системами спостереження й аналізу продуктивності.

3.6 Архітектура та структура пакетів проєкту

Розроблений серверний застосунок для управління системою оренди електросамокатів побудований за принципами чистої архітектури (Clean Architecture) з чітким розділенням обов'язків (Separation of Concerns). Використовується багатопшарова архітектура, що відображається у структурі пакетів Spring Boot проєкту.

Пакет **models** (або **entities**) містить класи, які є об'єктно-орієнтованим відображенням таблиць бази даних. До таких сутностей належать User, Scooter, Rental, Payment та Maintenance. Ці класи анотовані засобами JPA/Hibernate та не містять бізнес-логіки. Їх основне призначення — зберігання та передача даних між різними шарами застосунку.

Пакет **repositories** відповідає за шар доступу до даних (Data Access Layer). Він містить інтерфейси, які розширюють JpaRepository. Spring Data JPA автоматично генерує реалізацію цих інтерфейсів, надаючи можливість виконувати операції створення, читання, оновлення та видалення даних без написання SQL-запитів. Крім стандартних методів, репозиторії містять спеціалізовані запити для пошуку електросамокатів, отримання статистики оренд та аналізу використання транспорту.

Пакет **services** інкапсулює бізнес-логіку системи. Класи цього пакета анотовані як @Service та відповідають за реалізацію процесів оренди електросамокатів, перевірку їх доступності, обробку платежів, контроль технічного стану транспорту та виконання інших бізнес-операцій. Сервіси взаємодіють із репозиторіями та повертають оброблені результати контролерам. Такий підхід спрощує тестування та підтримку програмного забезпечення.

Пакет **controllers** представляє шар обробки HTTP-запитів. Контролери анотовані як @RestController та містять методи, що відповідають за

маршрутизацію запитів до REST API. Вони приймають дані від клієнтських застосунків у форматі JSON, передають їх відповідним сервісам та формують HTTP-відповіді з необхідними статус-кодами (200 OK, 201 Created, 404 Not Found тощо).

Додатково проєкт може містити пакет **dto** для передачі даних між клієнтом і сервером, пакет **config** для конфігурації безпеки та підключення до бази даних, а також пакет **exceptions** для централізованої обробки помилок. Така структура забезпечує масштабованість, зрозумілість коду та спрощує подальший розвиток системи оренди електросамокатів.

3.7 Обробка винятків та валідація вхідних даних

Для забезпечення стабільної роботи REST API та захисту системи оренди електросамокатів від некоректних даних з боку клієнтських застосунків реалізовано механізми валідації та централізованої обробки виняткових ситуацій.

Валідація даних виконується на рівні DTO-об'єктів (Data Transfer Objects) за допомогою стандарту Bean Validation та анотацій `@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Min` і `@Max`. Це дозволяє перевіряти коректність введених даних ще до їх передачі в сервісний шар. Наприклад, під час реєстрації користувача перевіряється правильність формату електронної пошти, а при додаванні електросамоката контролюється допустимий рівень заряду батареї та коректність основних параметрів транспортного засобу. У разі порушення правил валідації система автоматично повертає клієнту відповідь зі статусом 400 Bad Request.

Для централізованої обробки помилок використовується механізм Spring `@ControllerAdvice`. Створений клас `GlobalExceptionHandler` перехоплює всі винятки, що виникають під час роботи застосунку. Наприклад, якщо користувач намагається отримати інформацію про електросамокат або оренду за неіснуючим ідентифікатором, генерується виняток `EntityNotFoundException`.

Замість стандартних повідомлень про помилки Spring Framework система формує структуровану JSON-відповідь, яка містить текст помилки, час її виникнення (timestamp), код помилки та відповідний HTTP-статус (400 Bad Request, 404 Not Found, 500 Internal Server Error тощо). Такий підхід спрощує інтеграцію з веб- та мобільними клієнтами, а також полегшує процес налагодження та підтримки системи.

Реалізовані механізми валідації та обробки винятків підвищують надійність сервісу, забезпечують цілісність даних і гарантують коректну

взаємодію між клієнтськими застосунками та серверною частиною системи оренди електросамокатів.

ТЕСТУВАННЯ ТА НАЛАГОДЖЕННЯ

Для перевірки працездатності розробленого REST API системи оренди електросамокатів використовувалася консольна утиліта cURL, яка дозволяє виконувати HTTP-запити безпосередньо з командного рядка. Тестування проводилося шляхом відправлення запитів до локального сервера та аналізу отриманих відповідей у форматі JSON.

Процес додавання нового електросамоката перевірявся за допомогою методу POST. Сервер успішно обробив JSON-тіло запиту, зберіг об'єкт у базу даних та повернув статус 200 OK разом із призначеним ідентифікатором (id).

Запит на створення електросамоката:

```
curl -X POST http://localhost:8080/scooters \
-H "Content-Type: application/json" \
-d '{
  "qrCode": "SC001",
  "model": "Xiaomi Pro 2",
  "status": "AVAILABLE",
  "batteryLevel": 95,
  "location": "Odesa"
}'
```

Відповідь сервера:

```
{
  "id": 1,
  "qrCode": "SC001",
  "model": "Xiaomi Pro 2",
  "status": "AVAILABLE",
  "batteryLevel": 95,
  "location": "Odesa"
}
```

Далі було перевірено механізм пошуку електросамокатів. При зверненні до кінцевої точки /scooters/search?query=Xiaomi система успішно відфільтрувала результати в базі даних та повернула масив відповідних об'єктів.

Запит на пошук електросамоката:

```
curl -X GET "http://localhost:8080/scooters/search?query=Xiaomi"
```

Відповідь сервера:

```
[
  {
    "id": 1,
    "qrCode": "SC001",
    "model": "Xiaomi Pro 2",
    "status": "AVAILABLE",
    "batteryLevel": 95,
    "location": "Odesa"
  }
]
```

Окремо було протестовано роботу системних метрик сервісу, що генеруються модулем Spring Boot Actuator. При зверненні до кінцевої точки стану здоров'я застосунку система підтверджує свій статус, що свідчить про готовність серверної частини до роботи.

Запит для перевірки стану сервісу:

```
curl -X GET http://localhost:8080/actuator/health
```

Відповідь сервера:

```
{
  "status": "UP"
}
```

Результати тестування підтвердили коректну роботу REST API, правильне виконання CRUD-операцій, механізмів пошуку та системного моніторингу. Серверна частина успішно взаємодіє з базою даних PostgreSQL та забезпечує стабільну роботу системи оренди електросамокатів.

ВИСНОВКИ

У результаті виконання курсової роботи було спроектовано та реалізовано серверну частину програмного забезпечення для управління системою оренди електросамокатів.

Під час розробки було досягнуто наступних результатів:

- проведено системний аналіз предметної області, визначено основні сутності (User, Scooter, Rental, Payment, Maintenance) та зв'язки між ними;
- спроектовано об'єктно-орієнтовану модель бази даних із застосуванням принципів нормалізації та забезпечення цілісності даних;
- реалізовано повноцінний RESTful вебсервіс мовою Java з використанням фреймворку Spring Boot, який забезпечує управління користувачами, електросамокатами, орендами, платежами та технічним обслуговуванням;
- імплементовано механізми пошуку, фільтрації та отримання статистичних даних щодо використання електросамокатів;
- забезпечено моніторинг роботи серверного застосунку за допомогою Spring Boot Actuator та системних метрик;
- реалізовано механізми валідації даних та централізованої обробки виняткових ситуацій для підвищення надійності системи.

Усі поставлені завдання були виконані в повному обсязі. Працездатність розробленої системи підтверджена успішним тестуванням за допомогою консольної утиліти cURL та перевіркою коректності роботи REST API.

Під час виконання роботи було отримано практичні навички проєктування серверних застосунків, роботи з технологіями Spring Boot, Spring Data JPA, PostgreSQL, REST API та принципами побудови багаторівневої архітектури програмного забезпечення. Отримані знання можуть бути використані для подальшої розробки сучасних корпоративних та комерційних інформаційних систем.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Блінова Т. О., Коротєєва Т. О. Об'єктно-орієнтоване програмування : конспект лекцій. Львів : Видавництво Львівської політехніки, 2021. 184 с.
2. Еккель Б. Філософія Java. 4-те вид. Пітер, 2015. 1168 с.
3. Уоллс К. Spring у дії. 6-те вид. Київ : Форс Україна, 2023. 512 с.
4. Субраманіан С. PostgreSQL. Практичний посібник для розробників. Київ : Діалектика, 2022. 432 с.
5. Spring Data JPA Documentation. URL: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/> (дата звернення: 11.06.2026).

6. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/> (дата звернення: 11.06.2026).
7. cURL Documentation. URL: <https://curl.se/docs/> (дата звернення: 11.06.2026).
8. Spring Boot Documentation. URL: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (дата звернення: 11.06.2026).
9. Hibernate ORM Documentation. URL: <https://hibernate.org/orm/documentation/> (дата звернення: 11.06.2026).
10. Richardson L., Amundsen M., Ruby S. RESTful Web APIs. Sebastopol : O'Reilly Media, 2013. 408 p.
11.06.2026).

ДОДАТОК А

Лістинг вихідного коду програми

```
// =====  
// Файл конфігурації: application.properties  
// =====  
  
spring.datasource.url=jdbc:postgresql://localhost:5432/scooter_rental_db  
spring.datasource.username=postgres  
spring.datasource.password=root  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true  
  
management.endpoints.web.exposure.include=health,metrics,prometheus  
  
// =====  
// Головний клас: ScooterRentalApplication.java  
// =====  
  
package com.scooter.rental;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```

@SpringBootApplication
public class ScooterRentalApplication {

    public static void main(String[] args) {
        SpringApplication.run(ScooterRentalApplication.class, args);
    }
}

// =====
// Пакет model (Сутності БД)
// =====

// Файл: User.java

package com.scooter.rental.model;

import jakarta.persistence.*;
import java.util.List;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private String phone;

    @OneToMany(mappedBy = "user")
    private List<Rental> rentals;

    public User() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

```

```
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }
}
```

// Файл: Scooter.java

```
package com.scooter.rental.model;
```

```
import jakarta.persistence.*;
import java.util.List;
```

```
@Entity
@Table(name = "scooters")
public class Scooter {
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

```
private String qrCode;
private String model;
private Integer batteryLevel;
private String status;
private String location;
```

```
@OneToMany(mappedBy = "scooter")
private List<Rental> rentals;
```

```
public Scooter() {}
```

```
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
```

```
public String getQrCode() { return qrCode; }
public void setQrCode(String qrCode) { this.qrCode = qrCode; }
```

```
public String getModel() { return model; }
public void setModel(String model) { this.model = model; }
```



```
public Integer getBatteryLevel() { return batteryLevel; }  
public void setBatteryLevel(Integer batteryLevel) {  
    this.batteryLevel = batteryLevel;  
}
```

```
public String getStatus() { return status; }  
public void setStatus(String status) { this.status = status; }
```

```
public String getLocation() { return location; }  
public void setLocation(String location) {  
    this.location = location;  
}  
}
```

// Файл: Rental.java

```
package com.scooter.rental.model;
```

```
import jakarta.persistence.*;  
import java.time.LocalDateTime;
```

```
@Entity  
@Table(name = "rentals")  
public class Rental {
```

```
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;
```

```
    private LocalDateTime startTime;  
    private LocalDateTime endTime;
```

```
    private Double totalPrice;
```

```
    @ManyToOne  
    @JoinColumn(name = "user_id")  
    private User user;
```

```
    @ManyToOne  
    @JoinColumn(name = "scooter_id")  
    private Scooter scooter;
```

```
public Rental() {}

public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public LocalDateTime getStartTime() {
return startTime;
}

public void setStartTime(LocalDateTime startTime) {
this.startTime = startTime;
}

public LocalDateTime getEndTime() {
return endTime;
}

public void setEndTime(LocalDateTime endTime) {
this.endTime = endTime;
}

public Double getTotalPrice() {
return totalPrice;
}

public void setTotalPrice(Double totalPrice) {
this.totalPrice = totalPrice;
}
}
```

// Файл: Payment.java

```
package com.scooter.rental.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
@Table(name = "payments")
```

```
public class Payment {
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private Double amount;
private String paymentMethod;

@OneToOne
@JoinColumn(name = "rental_id")
private Rental rental;
}
```

// Файл: Maintenance.java

```
package com.scooter.rental.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
@Table(name = "maintenance")
public class Maintenance {
```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

```
private String type;
private String description;
```

```
@ManyToOne
@JoinColumn(name = "scooter_id")
private Scooter scooter;
}
```

